POLY PMNA · DIPLOMA CURRICULUM REVISION 2026

Course 6041T | Semester VI | Theory | 4 Credits

Verilog Programming & Applications

A complete syllabus-aligned handbook with clear explanations, practical or exam guidance, Malayalam-support notes, diagrams, activities and revision tools.

Programmes	**Electronics Engineering**
Contact hours	**60**
Teaching scheme	**3:1:0:0**
Credits	**4**
CIE / ESE	**Theory CIE 40 / Theory ESE 60**
Self-learning	**5.5 h/week**

CURRICULUM INTEGRITY

Official Source Declaration and Verified Inconsistencies

The attached Revision 2026 index identifies Course 6041T as Verilog Programming & Applications in Semester VI for Electronics Engineering. The official SITTTR detailed course page was checked at the indexed link and returned “Requested file not found.” With the user's approved public-source approach, this handbook is transparently based on NPTEL IIT Guwahati’s Digital Design with Verilog course and polytechnic Verilog curricula. It does not claim to reproduce official REV2026 detailed-syllabus wording.

Source treatment: Teaching scheme, credits and assessment values are provisional placeholders aligned to neighbouring handbook format. Verilog designs, FSM logic, FPGA constraints and digital circuits must be based on approved engineering plans, certified hardware data, current timing standards and competent professional review.

Verified source note: Verified sources support Verilog syntax, modeling styles, FSM design, and verification techniques. The four-module sequence is a learning path, not an asserted official module split.

COURSE DASHBOARD

What You Are Expected to Learn

60

Contact hours

4

Credits

Theory CIE 40

CIE

Theory ESE 60

ESE

Course introduction

Verilog Programming & Applications studies the modeling, simulation and synthesis of digital systems using Hardware Description Language (HDL). It develops the ability to write Verilog code using various modeling styles (Gate, Dataflow, Behavioral), design complex combinational and sequential circuits, implement Finite State Machines (FSM), and coordinate FPGA-based digital solutions while respecting the technical and timing boundaries of electronic and VLSI engineering.

> **Malayalam support:** □ handbook □□□□□□□□□□□□ syllabus topic □□□□□ □□□□□□□□□□; □□□□□□□□ principle, diagram/procedure, application, common mistake □□□□□□ □□□□□□□□ □□□□□□□□.

Prerequisite foundation

Digital electronics, computer organization, and basic C programming logic.

Use prerequisites only as a revision bridge. The current course outcomes and detailed syllabus define the required performance.

Teaching and assessment scheme

L	T	P	S	Self-learning/week	Contact hours	Credits	CIE	ESE
3	1	0	0	5.5 h/week	60	4	Theory CIE 40	Theory ESE 60

STUDY METHOD

How to Use This Handbook

1 · Understand

Read terms, conditions and purpose; make a short definition in your own words.

2 · Visualise

Follow the diagram, circuit, flow or procedure and label it accurately.

3 · Apply

Use the method block, calculation or practical checklist without skipping units or safety checks.

4 · Revise

Use questions, quick cards and common-mistake notes before examination.

OFFICIAL STATEMENTS

Objectives and Course Outcomes

Official course objectives

1. Explain the design methodology and syntax of Verilog HDL for digital system modeling.
2. Apply various Verilog modeling styles to implement combinational and sequential digital circuits.
3. Analyze and design Finite State Machines (FSM) using behavioral Verilog constructs.
4. Apply verification techniques using testbenches and understand the FPGA synthesis and implementation flow.

Student-friendly meaning

You should be able to recognise the relevant system or material, explain its principle, communicate through a correct diagram/table where needed and follow a defensible solution or practical method.

> **Malayalam support:** CO □□□□□□ exam-□ □□□□□ □□□□□□□ □□□□□□ □□□□□ □□□□□□□□. Define, explain, apply □□□□□ action words □□□□□□□□□□□□.

Official Course Outcomes

CO	Official outcome	Bloom level
CO1	Explain the fundamentals and syntax of Verilog HDL modeling.	Understand
CO2	Develop Verilog models for combinational and sequential digital circuits.	Apply
CO3	Design and implement Finite State Machines (FSM) using Verilog.	Apply
CO4	Verify digital designs using testbenches and implement on FPGA/CPLD.	Apply

Official CO–PO articulation matrix							
CO	**PO1**	**PO2**	**PO3**	**PO4**	**PO5**	**PO6**	**PO7–PO11**
CO1	3	2	2	2	2	–	–
CO2	3	2	2	2	2	–	–
CO3	3	2	2	2	2	–	–
CO4	3	2	2	2	2	–	–

SYLLABUS COVERAGE

Curriculum Coverage Map

All official major topics mapped to handbook chapters					
Module	**Title**	**Official major topics**	**Hours**	**CO**	**Handbook section**
1	Verilog HDL Fundamentals	Introduction to HDL; Design methodology: Top-down and Bottom-up; Verilog design flow; Lexical conventions; Data types: Nets, Registers, Vectors; Modules and Ports.	Approx. 14 hours	CO1	Module 1
2	Verilog Modeling Styles	Gate-level modeling: Primitives, Delays; Dataflow modeling: Continuous assignments (assign), Operators; Behavioral modeling: initial and always blocks; Procedural assignments: Blocking (=) vs Non-blocking (<=).	Approx. 16 hours	CO2	Module 2
3	Digital Design and State Machines	Modeling combinational circuits: ALU, Decoders, Encoders; Modeling sequential circuits: Shift registers, Counters; Finite State Machines (FSM): Moore and Mealy models; FSM design using Verilog.	Approx. 16 hours	CO3	Module 3
4	Verification and FPGA Implementation	Writing Testbenches: Stimulus generation, Monitoring; System tasks: $display, $monitor, $stop, $finish; Tasks and Functions; FPGA/CPLD architecture; Synthesis flow and implementation.	Approx. 14 hours	CO4	Module 4

LEARNING ROADMAP

How the Modules Connect

Verilog HDL Fundamentals

Introduction to HDL; Design methodology: Top-down and Bottom-up; Verilog design flow; Lexical conventions; Data types: Nets, Registers, Vectors; Modules and Ports.

CO1 · Approx. 14 hours

Verilog Modeling Styles

Gate-level modeling: Primitives, Delays; Dataflow modeling: Continuous assignments (assign), Operators; Behavioral modeling: initial and always blocks; Procedural assignments: Blocking (=) vs Non-blocking (<=).

CO2 · Approx. 16 hours

Digital Design and State Machines

Modeling combinational circuits: ALU, Decoders, Encoders; Modeling sequential circuits: Shift registers, Counters; Finite State Machines (FSM): Moore and Mealy models; FSM design using Verilog.

CO3 · Approx. 16 hours

Verification and FPGA Implementation

Writing Testbenches: Stimulus generation, Monitoring; System tasks: $display, $monitor, $stop, $finish; Tasks and Functions; FPGA/CPLD architecture; Synthesis flow and implementation.

CO4 · Approx. 14 hours

MODULE 1

Verilog HDL Fundamentals

Introduction to HDL; Design methodology: Top-down and Bottom-up; Verilog design flow; Lexical conventions; Data types: Nets, Registers, Vectors; Modules and Ports.

Approx. 14 hours **CO1** **Understand · Apply**

Learning goals

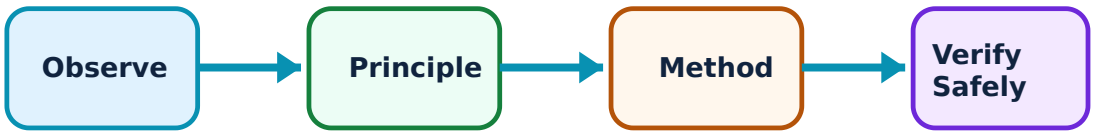
Identify core terms, explain the principle and complete the stated application or practical outcome.

Key evidence

Use a labelled diagram, table, graph, procedure or calculation that matches the topic.

Engineering habit

Check units, polarity/direction, ratings, materials and safety before concluding.



Study flow for Verilog HDL Fundamentals: identify the system, state its principle, follow the correct method and verify the result safely.

1. HDL Introduction and Flow–

Hardware Description Languages (HDL) allow for the textual description of digital hardware. Verilog is a popular HDL used for modeling, simulation, and synthesis. The design flow includes design entry, functional simulation, synthesis, and timing verification.

Study and application method

Differentiate between 'Modeling' and 'Synthesis'; describe the steps in a typical 'Verilog Design Flow'.

Connect the explanation to the official student learning outcome before attempting a numerical, diagram, procedure or viva answer.

Evidence of understanding

Use correct terminology, a labelled sketch/table where useful, correct units or conditions, and a conclusion linked to the observed or calculated result.

How to construct a complete answer

Given: identify the quantities, device condition, waveform or circuit arrangement.

Required: state exactly what must be found or explained.

Method: Differentiate between 'Modeling' and 'Synthesis'; describe the steps in a typical 'Verilog Design Flow'.

Answer habit: show a labelled step, the relevant unit/condition and a short interpretation.

Malayalam support: □□□□□ concept □□□□□□□ □□□□□ □□□□□□□□□□□□□. □□□□ diagram / circuit / formula / procedure □□□□□□□□□□□ □□□□□□□□. □□□□□□□□□ result □□□□□□□□□□ application □□□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□□□. Technical terms English-□ □□□□□ □□□□□□□□□□□□.

Common mistake / precaution: Verilog code for simulation may not always be synthesizable. Do not use non-synthesizable constructs (like #delays) in code intended for actual hardware implementation.

2. Syntax and Data Objects−

Verilog uses nets (like 'wire') to represent physical connections and registers (like 'reg') to store values in procedural blocks. Lexical conventions include keywords, identifiers, and comments. Modules are the basic building blocks, communicating through defined ports.

Study and application method

Explain the difference between 'wire' and 'reg' data types; sketch the structure of a 'Verilog Module'.

Connect the explanation to the official student learning outcome before attempting a numerical, diagram, procedure or viva answer.

Evidence of understanding

Use correct terminology, a labelled sketch/table where useful, correct units or conditions, and a conclusion linked to the observed or calculated result.

How to construct a complete answer

Given: identify the quantities, device condition, waveform or circuit arrangement.

Required: state exactly what must be found or explained.

Method: Explain the difference between 'wire' and 'reg' data types; sketch the structure of a 'Verilog Module'.

Answer habit: show a labelled step, the relevant unit/condition and a short interpretation.

Malayalam support: □□□□□ concept □□□□□□□ □□□□□ □□□□□□□□□□□□□. □□□□ diagram / circuit / formula / procedure □□□□□□□□□□□ □□□□□□□□. □□□□□□□□□ result □□□□□□□□□□ application □□□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□□□. Technical terms English-□ □□□□□ □□□□□□□□□□□□.

Common mistake / precaution: Port mismatch can lead to simulation errors or synthesis failure. All module instantiations must follow authorised port mapping and naming conventions.

Module 1 revision cue: Re-read official major topics, redraw one key diagram/flow and explain one practical or numerical method without looking at notes.

MODULE 2

Verilog Modeling Styles

Gate-level modeling: Primitives, Delays; Dataflow modeling: Continuous assignments (assign), Operators; Behavioral modeling: initial and always blocks; Procedural assignments: Blocking (=) vs Non-blocking (<=).

Approx. 16 hours | CO2 | Understand · Apply

Learning goals

Identify core terms, explain the principle and complete the stated application or practical

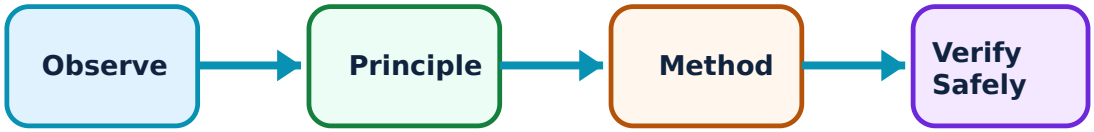
outcome.

Key evidence

Use a labelled diagram, table, graph, procedure or calculation that matches the topic.

Engineering habit

Check units, polarity/direction, ratings, materials and safety before concluding.



Study flow for Verilog Modeling Styles: identify the system, state its principle, follow the correct method and verify the result safely.

1. Gate and Dataflow Modeling–

Gate-level modeling uses built-in primitives like and, or, not. Dataflow modeling uses the 'assign' statement to describe the flow of data through the circuit using operators. These styles are suitable for simple combinational logic.

Study and application method

Write a gate-level Verilog model for a '2-to-1 Multiplexer'; explain the 'assign' statement in dataflow modeling.

Connect the explanation to the official student learning outcome before attempting a numerical, diagram, procedure or viva answer.

Evidence of understanding

Use correct terminology, a labelled sketch/table where useful, correct units or conditions, and a conclusion linked to the observed or calculated result.

How to construct a complete answer

Given: identify the quantities, device condition, waveform or circuit arrangement.

Required: state exactly what must be found or explained.

Method: Write a gate-level Verilog model for a '2-to-1 Multiplexer'; explain the 'assign' statement in dataflow modeling.

Answer habit: show a labelled step, the relevant unit/condition and a short interpretation.

Malayalam support: □□□□□ concept □□□□□□□ □□□□□ □□□□□□□□□□□□□. □□□□ diagram / circuit / formula / procedure □□□□□□□□□□□ □□□□□□□□. □□□□□□□□□ result □□□□□□□□□□ application □□□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□□□. Technical terms English-□ □□□□□ □□□□□□□□□□□.

Common mistake / precaution: Complex gate-level models are hard to maintain. Do not use manual gate-level entry for large designs; prefer dataflow or behavioral modeling for better abstraction.

2. Behavioral Modeling–

Behavioral modeling describes the algorithm or behavior of the circuit using procedural blocks ('always' and 'initial'). It supports control statements like if-else, case, and loops. This style is essential for modeling complex sequential logic and state machines.

Study and application method

Compare 'Blocking' and 'Non-blocking' assignments with examples; write a behavioral model for a 'D Flip-flop' with reset.

Connect the explanation to the official student learning outcome before attempting a numerical, diagram, procedure or viva answer.

Evidence of understanding

Use correct terminology, a labelled sketch/table where useful, correct units or conditions, and a conclusion linked to the observed or calculated result.

How to construct a complete answer

Given: identify the quantities, device condition, waveform or circuit arrangement.

Required: state exactly what must be found or explained.

Method: Compare 'Blocking' and 'Non-blocking' assignments with examples; write a behavioral model for a 'D Flip-flop' with reset.

Answer habit: show a labelled step, the relevant unit/condition and a short interpretation.

Malayalam support: □□□□□ concept □□□□□□□ □□□□□ □□□□□□□□□□□□□. □□□□ diagram / circuit / formula / procedure □□□□□□□□□□□ □□□□□□□□. □□□□□□□□□ result □□□□□□□□□□ application □□□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□□□. Technical terms English-□ □□□□□ □□□□□□□□□□□.

Common mistake / precaution: Incorrect sensitivity lists in 'always' blocks can cause simulation-synthesis mismatches. Always include all required signals in the sensitivity list as per authorised coding standards.

Module 2 revision cue: Re-read official major topics, redraw one key diagram/flow and explain one practical or numerical method without looking at notes.

MODULE 3

Digital Design and State Machines

Modeling combinational circuits: ALU, Decoders, Encoders; Modeling sequential circuits: Shift registers, Counters; Finite State Machines (FSM): Moore and Mealy models; FSM design using Verilog.

Approx. 16 hours **CO3** **Understand · Apply**

Learning goals

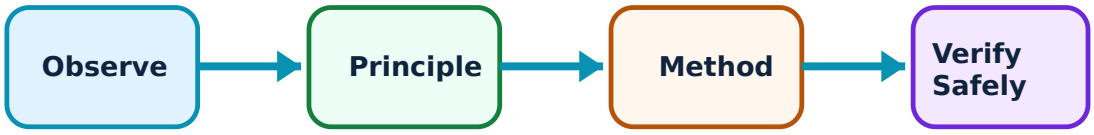
Identify core terms, explain the principle and complete the stated application or practical outcome.

Key evidence

Use a labelled diagram, table, graph, procedure or calculation that matches the topic.

Engineering habit

Check units, polarity/direction, ratings, materials and safety before concluding.



Study flow for Digital Design and State Machines: identify the system, state its principle, follow the correct method and verify the result safely.

1. Combinational and Sequential Design–

Verilog is used to model standard digital components. An ALU can be modeled using a 'case' statement. Sequential circuits like counters use 'always' blocks triggered by the clock edge. Parameters are used to create reusable, bit-width independent modules.

Study and application method

Develop a Verilog model for a '4-bit Binary Counter'; explain the use of 'parameter' for designing generic modules.

Connect the explanation to the official student learning outcome before attempting a numerical, diagram, procedure or viva answer.

Evidence of understanding

Use correct terminology, a labelled sketch/table where useful, correct units or conditions, and a conclusion linked to the observed or calculated result.

How to construct a complete answer

Given: identify the quantities, device condition, waveform or circuit arrangement.

Required: state exactly what must be found or explained.

Method: Develop a Verilog model for a '4-bit Binary Counter'; explain the use of 'parameter' for designing generic modules.

Answer habit: show a labelled step, the relevant unit/condition and a short interpretation.

Malayalam support: □□□□□ concept □□□□□□□ □□□□□ □□□□□□□□□□□□□. □□□□ diagram / circuit / formula / procedure □□□□□□□□□□□ □□□□□□□□. □□□□□□□□□ result □□□□□□□□□□ application □□□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□□□. Technical terms English-□ □□□□□ □□□□□□□□□□□.

Common mistake / precaution: Asynchronous resets can cause timing issues if not handled carefully. All sequential designs must follow authorised clocking and reset strategies.

2. Finite State Machines (FSM)–

FSMs are used to model complex control logic. Moore machines have outputs depending only on the current state, while Mealy machine outputs depend on both state and inputs. In Verilog, FSMs are typically implemented using a state register and two procedural blocks (state transition and output logic).

Study and application method

Sketch the state diagram for a 'Sequence Detector' (e.g., 1011); write the behavioral Verilog code for a 'Moore FSM'.

Connect the explanation to the official student learning outcome before attempting a numerical, diagram, procedure or viva answer.

Evidence of understanding

Use correct terminology, a labelled sketch/table where useful, correct units or conditions, and a conclusion linked to the observed or calculated result.

How to construct a complete answer

Given: identify the quantities, device condition, waveform or circuit arrangement.

Required: state exactly what must be found or explained.

Method: Sketch the state diagram for a 'Sequence Detector' (e.g., 1011); write the behavioral Verilog code for a 'Moore FSM'.

Answer habit: show a labelled step, the relevant unit/condition and a short interpretation.

Malayalam support: □□□□□ concept □□□□□□□ □□□□□ □□□□□□□□□□□□□. □□□□ diagram / circuit / formula / procedure □□□□□□□□□□□ □□□□□□□□. □□□□□□□□□ result □□□□□□□□□□ application □□□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□□□. Technical terms English-□ □□□□□ □□□□□□□□□□□.

Common mistake / precaution: Unreachable states or deadlocks in FSMs can hang the system. All FSM designs must include authorised default/reset states and be verified for complete state coverage.

Module 3 revision cue: Re-read official major topics, redraw one key diagram/flow and explain one practical or numerical method without looking at notes.

MODULE 4

Verification and FPGA Implementation

Writing Testbenches: Stimulus generation, Monitoring; System tasks: $display, $monitor, $stop, $finish; Tasks and Functions; FPGA/CPLD architecture; Synthesis flow and implementation.

Approx. 14 hours **CO4** **Understand · Apply**

Learning goals

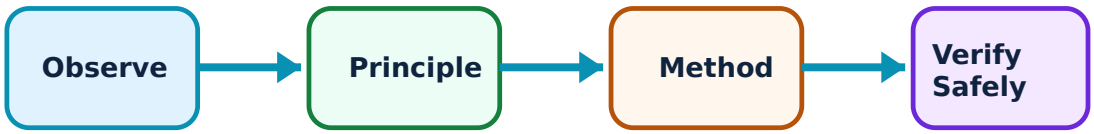
Identify core terms, explain the principle and complete the stated application or practical outcome.

Key evidence

Use a labelled diagram, table, graph, procedure or calculation that matches the topic.

Engineering habit

Check units, polarity/direction, ratings, materials and safety before concluding.



Study flow for Verification and FPGA Implementation: identify the system, state its principle, follow the correct method and verify the result safely.

1. Verification using Testbenches–

A testbench is a non-synthesizable Verilog module used to apply stimulus to the design under test (DUT) and monitor its response. System tasks help in displaying simulation results and controlling the simulation run.

Study and application method

Explain the purpose of a 'Testbench'; write a simple testbench to verify a 'Full Adder' module.

Connect the explanation to the official student learning outcome before attempting a numerical, diagram, procedure or viva answer.

Evidence of understanding

Use correct terminology, a labelled sketch/table where useful, correct units or conditions, and a conclusion linked to the observed or calculated result.

How to construct a complete answer

Given: identify the quantities, device condition, waveform or circuit arrangement.

Required: state exactly what must be found or explained.

Method: Explain the purpose of a 'Testbench'; write a simple testbench to verify a 'Full Adder' module.

Answer habit: show a labelled step, the relevant unit/condition and a short interpretation.

Malayalam support: □□□□□ concept □□□□□□□ □□□□□ □□□□□□□□□□□□□. □□□□ diagram / circuit / formula / procedure □□□□□□□□□□□ □□□□□□□□. □□□□□□□□□ result □□□□□□□□□□ application □□□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□□. Technical terms English-□ □□□□□ □□□□□□□□□□□.

Common mistake / precaution: Incomplete testbenches can miss critical design bugs. All designs must be verified with authorised test plans covering all edge cases and functional scenarios.

2. FPGA Synthesis and Implementation–

Synthesis converts the HDL code into a gate-level netlist based on the target FPGA/CPLD library. Implementation involves mapping, placing, and routing the netlist onto the physical hardware. Timing analysis ensures the design meets the required clock frequency.

Study and application method

Describe the 'FPGA Synthesis' process; explain the significance of 'Timing Constraints' in FPGA design.

Connect the explanation to the official student learning outcome before attempting a numerical, diagram, procedure or viva answer.

Evidence of understanding

Use correct terminology, a labelled sketch/table where useful, correct units or conditions, and a conclusion linked to the observed or calculated result.

How to construct a complete answer

Given: identify the quantities, device condition, waveform or circuit arrangement.

Required: state exactly what must be found or explained.

Method: Describe the 'FPGA Synthesis' process; explain the significance of 'Timing Constraints' in FPGA design.

Answer habit: show a labelled step, the relevant unit/condition and a short interpretation.

Malayalam support: □□□□□ concept □□□□□□□ □□□□□ □□□□□□□□□□□□□. □□□□ diagram / circuit / formula / procedure □□□□□□□□□□□ □□□□□□□□. □□□□□□□□□ result □□□□□□□□□□ application □□□□□□□ □□□□□□□□□□ □□□□□ □□□□□□□□□□□□□. Technical terms English-□ □□□□□ □□□□□□□□□□□.

Common mistake / precaution: FPGA implementation results depend on the toolchain and device architecture. Do not deploy designs on hardware without authorised timing closure and power analysis.

Module 4 revision cue: Re-read official major topics, redraw one key diagram/flow and explain one practical or numerical method without looking at notes.

FORMULA AND PROCEDURE BANK

Rapid Recall Bank

Recall 1

State symbols and SI units before substitution.

Recall 2

Draw the circuit, system boundary, vector or process flow before reasoning.

Recall 3

> For laboratory work: aim → apparatus → diagram → procedure → observation → calculation → result → precautions.

Recall 4

> For a graph: label axes, units, scale, operating region and conclusion.

Recall 5

> For a comparison: use construction, working, result, advantage and limitation.

Recall 6

> For an extended answer: definition/principle, labelled diagram, working steps, application and a caution/limitation.

DIAGRAM LIBRARY

Diagram Library for Rapid Revision

Module 1: Verilog HDL Fundamentals

Use the concept flow to revise observation, principle, method and verification.

Module 2: Verilog Modeling Styles

Use the concept flow to revise observation, principle, method and verification.

Module 3: Digital Design and State Machines

Use the concept flow to revise observation, principle, method and verification.

Module 4: Verification and FPGA Implementation

Use the concept flow to revise observation, principle, method and verification.

SELF-LEARNING

Official Self-Learning and Student Activities

Structured activity

Write a Verilog module for a Full Adder using dataflow modeling.

Keep evidence in your record: date, source/observation, diagram/table, key learning and questions for discussion.

Structured activity

Develop a behavioral Verilog code for a 4-bit Shift Register with parallel load.

Keep evidence in your record: date, source/observation, diagram/table, key learning and questions for discussion.

Structured activity

Draw the state diagram and write the Verilog code for a 3-bit Gray code counter.

Keep evidence in your record: date, source/observation, diagram/table, key learning and questions for discussion.

Structured activity

Create a simple testbench to verify a 2-to-4 Decoder module.

Keep evidence in your record: date, source/observation, diagram/table, key learning and questions for discussion.

Structured activity

Identify a digital system (e.g., Vending machine controller) and sketch its FSM state diagram.

Keep evidence in your record: date, source/observation, diagram/table, key learning and questions for discussion.

Activity discipline: The supplied PDF assigns the listed self-learning hours. This handbook preserves the activity direction; the teacher’s approved schedule and safety instructions govern execution.

EXAMINATION READINESS

Assessment Methodology and Examination Pattern

Official assessment structure		
Component	**Official mark / conversion**	**Student evidence**
Attendance	5 marks	End-of-semester attendance component.
Self-learning activities	15 marks	Module-linked activities.
Series examinations	20 + 20 converted	Two 50-mark series examinations.
CIE total	Theory CIE 40	Continuous internal evaluation.
Written ESE	Theory ESE 60	2.5 hours; official Part A / Part B / Part C pattern.

Series-test pattern

The supplied theory pattern uses Part A (1-mark), Part B (3-mark with choice) and Part C (7-mark) distribution across the stated modules. Practical courses follow the supplied practical-test scheme.

Examination evidence

Show a complete method, correct units/conditions, clear diagrams or tables, a result/conclusion and safety awareness for any apparatus or process involved.

EXAM PRACTICE

Module-wise Questions with Answers

Module 1 — Verilog HDL Fundamentals

Explain HDL Introduction and Flow and state one correct method, application or precaution.–

Hardware Description Languages (HDL) allow for the textual description of digital hardware. Verilog is a popular HDL used for modeling, simulation, and synthesis. The design flow includes design entry, functional simulation, synthesis, and timing verification.

Answer framework: Differentiate between 'Modeling' and 'Synthesis'; describe the steps in a typical 'Verilog Design Flow'.

Important point: Verilog code for simulation may not always be synthesizable. Do not use non-synthesizable constructs (like #delays) in code intended for actual hardware implementation.

Explain Syntax and Data Objects and state one correct method, application or precaution.–

Verilog uses nets (like 'wire') to represent physical connections and registers (like 'reg') to store values in procedural blocks. Lexical conventions include keywords, identifiers, and comments. Modules are the basic building blocks, communicating through defined ports.

Answer framework: Explain the difference between 'wire' and 'reg' data types; sketch the structure of a 'Verilog Module'.

Important point: Port mismatch can lead to simulation errors or synthesis failure. All module instantiations must follow authorised port mapping and naming conventions.

Module 2 — Verilog Modeling Styles

Explain Gate and Dataflow Modeling and state one correct method, application or precaution.–

Gate-level modeling uses built-in primitives like and, or, not. Dataflow modeling uses the 'assign' statement to describe the flow of data through the circuit using operators. These styles are suitable for simple combinational logic.

Answer framework: Write a gate-level Verilog model for a '2-to-1 Multiplexer'; explain the 'assign' statement in dataflow modeling.

Important point: Complex gate-level models are hard to maintain. Do not use manual gate-level entry for large designs; prefer dataflow or behavioral modeling for better abstraction.

Explain Behavioral Modeling and state one correct method, application or precaution.

–

Behavioral modeling describes the algorithm or behavior of the circuit using procedural blocks ('always' and 'initial'). It supports control statements like if-else, case, and loops. This style is essential for modeling complex sequential logic and state machines.

Answer framework: Compare 'Blocking' and 'Non-blocking' assignments with examples; write a behavioral model for a 'D Flip-flop' with reset.

Important point: Incorrect sensitivity lists in 'always' blocks can cause simulation-synthesis mismatches. Always include all required signals in the sensitivity list as per authorised coding standards.

Module 3 — Digital Design and State Machines

Explain Combinational and Sequential Design and state one correct method, application or precaution.–

Verilog is used to model standard digital components. An ALU can be modeled using a 'case' statement. Sequential circuits like counters use 'always' blocks triggered by the clock edge. Parameters are used to create reusable, bit-width independent modules.

Answer framework: Develop a Verilog model for a '4-bit Binary Counter'; explain the use of 'parameter' for designing generic modules.

Important point: Asynchronous resets can cause timing issues if not handled carefully. All sequential designs must follow authorised clocking and reset strategies.

Explain Finite State Machines (FSM) and state one correct method, application or precaution.–

FSMs are used to model complex control logic. Moore machines have outputs depending only on the current state, while Mealy machine outputs depend on both state and inputs. In Verilog, FSMs are typically implemented using a state register and two procedural blocks (state transition and output logic).

Answer framework: Sketch the state diagram for a 'Sequence Detector' (e.g., 1011); write the behavioral Verilog code for a 'Moore FSM'.

Important point: Unreachable states or deadlocks in FSMs can hang the system. All FSM designs must include authorised default/reset states and be verified for complete state coverage.

Module 4 — Verification and FPGA Implementation

Explain Verification using Testbenches and state one correct method, application or precaution.–

A testbench is a non-synthesizable Verilog module used to apply stimulus to the design under test (DUT) and monitor its response. System tasks help in displaying simulation results and controlling the simulation run.

Answer framework: Explain the purpose of a 'Testbench'; write a simple testbench to verify a 'Full Adder' module.

Important point: Incomplete testbenches can miss critical design bugs. All designs must be verified with authorised test plans covering all edge cases and functional scenarios.

Explain FPGA Synthesis and Implementation and state one correct method, application or precaution.–

Synthesis converts the HDL code into a gate-level netlist based on the target FPGA/CPLD library. Implementation involves mapping, placing, and routing the netlist onto the physical hardware. Timing analysis ensures the design meets the required clock frequency.

Answer framework: Describe the 'FPGA Synthesis' process; explain the significance of 'Timing Constraints' in FPGA design.

Important point: FPGA implementation results depend on the toolchain and device architecture. Do not deploy designs on hardware without authorised timing closure and power analysis.

PRACTICE ONLY

Practice Model Paper

> **Important:** This practice paper is based on the official pattern in the supplied syllabus. It is **not** an official SITTTR model question paper.

Part A — short response

1. State one key definition from Module 1: Verilog HDL Fundamentals.
2. Identify one diagram, observation, formula or safety condition relevant to Module 1.
3. State one key definition from Module 2: Verilog Modeling Styles.
4. Identify one diagram, observation, formula or safety condition relevant to Module 2.
5. State one key definition from Module 3: Digital Design and State Machines.
6. Identify one diagram, observation, formula or safety condition relevant to Module 3.
7. State one key definition from Module 4: Verification and FPGA Implementation.

8. Identify one diagram, observation, formula or safety condition relevant to Module 4.

Part B — structured explanation

1. Explain a selected procedure/principle from Module 1 with a labelled diagram, table or method.
2. Explain a selected procedure/principle from Module 2 with a labelled diagram, table or method.
3. Explain a selected procedure/principle from Module 3 with a labelled diagram, table or method.
4. Explain a selected procedure/principle from Module 4 with a labelled diagram, table or method.

Part C — extended response / practical task

1. Develop a complete answer or practical plan for: Introduction to HDL; Design methodology: Top-down and Bottom-up; Verilog design flow; Lexical conventions; Data types: Nets, Registers, Vectors; Modules and Ports..
2. Develop a complete answer or practical plan for: Gate-level modeling: Primitives, Delays; Dataflow modeling: Continuous assignments (assign), Operators; Behavioral modeling: initial and always blocks; Procedural assignments: Blocking (=) vs Non-blocking (<=)..
3. Develop a complete answer or practical plan for: Modeling combinational circuits: ALU, Decoders, Encoders; Modeling sequential circuits: Shift registers, Counters; Finite State Machines (FSM): Moore and Mealy models; FSM design using Verilog..
4. Develop a complete answer or practical plan for: Writing Testbenches: Stimulus generation, Monitoring; System tasks: $display, $monitor, $stop, $finish; Tasks and Functions; FPGA/CPLD architecture; Synthesis flow and implementation..

LAST-MILE REVISION

Quick Revision

Module 1: Verilog HDL Fundamentals

Introduction to HDL; Design methodology: Top-down and Bottom-up; Verilog design flow; Lexical conventions; Data types: Nets, Registers, Vectors; Modules and Ports.

- Match each topic to CO1.
- Redraw or rehearse one key method.
- Check one common mistake/precaution.

Module 2: Verilog Modeling Styles

Gate-level modeling: Primitives, Delays; Dataflow modeling: Continuous assignments (assign), Operators; Behavioral modeling: initial and always blocks; Procedural assignments: Blocking (=) vs Non-blocking (<=).

- Match each topic to CO2.
- Redraw or rehearse one key method.
- Check one common mistake/precaution.

Module 3: Digital Design and State Machines

Modeling combinational circuits: ALU, Decoders, Encoders; Modeling sequential circuits: Shift registers, Counters; Finite State Machines (FSM): Moore and Mealy models; FSM design using Verilog.

- Match each topic to CO3.
- Redraw or rehearse one key method.
- Check one common mistake/precaution.

Module 4: Verification and FPGA Implementation

Writing Testbenches: Stimulus generation, Monitoring; System tasks: $display, $monitor, $stop, $finish; Tasks and Functions; FPGA/CPLD architecture; Synthesis flow and implementation.

- Match each topic to CO4.
- Redraw or rehearse one key method.
- Check one common mistake/precaution.

Common mistakes: Writing an unlabelled diagram; ignoring units, polarity or conditions; treating a practical result as valid without observations; confusing a definition with an application; and omitting a safety precaution where the topic uses apparatus, current, heat, chemicals or tools.

Exam checklist: Read the command word; answer every part; draw clean labelled diagrams; use a clear calculation/procedure; state result with units or observation; and reserve two minutes for checking.

VOCABULARY

Glossary

Important terms from the syllabus	
Term / topic	**One-line meaning**
HDL Introduction and Flow	Hardware Description Languages (HDL) allow for the textual description of digital hardware.
Syntax and Data Objects	Verilog uses nets (like 'wire') to represent physical connections and registers (like 'reg') to store values in procedural blocks.
Gate and Dataflow Modeling	Gate-level modeling uses built-in primitives like and, or, not.
Behavioral Modeling	Behavioral modeling describes the algorithm or behavior of the circuit using procedural blocks ('always' and 'initial').
Combinational and Sequential Design	Verilog is used to model standard digital components.
Finite State Machines (FSM)	FSMs are used to model complex control logic.
Verification using Testbenches	A testbench is a non-synthesizable Verilog module used to apply stimulus to the design under test (DUT) and monitor its response.
FPGA Synthesis and Implementation	Synthesis converts the HDL code into a gate-level netlist based on the target FPGA/CPLD library.

LEARNING RESOURCES

References

Recommended Verilog HDL references

1. Samir Palnitkar, Verilog HDL: A Guide to Digital Design and Synthesis.
2. J. Bhasker, A Verilog HDL Primer.
3. Michael D. Ciletti, Advanced Digital Design with the Verilog HDL.
4. T. R. Padmanabhan and B. Bala Tripura Sundari, Design through Verilog HDL.

Approved public Verilog HDL sources

- https://nptel.ac.in/courses/106103358
- https://nptel.ac.in/courses/108103179

These sources provide the verified study basis while official Course 6041T detail is unavailable; they do not replace official chip designs, authorised FPGA bitstreams, certified equipment specifications or authorised electronic product plans.